

CSC 1202 — Likely 2026 Exam (ILES Case Study)

Format: patterned exactly on the 2025 AITS paper. **Instructions to yourself when practising:**

1. Read the case study below first.
 2. Attempt any **five (5)** of the ten questions.
 3. Time yourself: 3 hours total, so ~36 minutes per question.
 4. **Cover the model answer with a blank sheet** while you answer. Only peek after writing.
 5. Mark your own paper line-by-line. Be ruthless: a missing `:` or `;` or wrong indent costs marks.
-

Case Study: Internship Logging & Evaluation System (ILES)

Makerere University has commissioned your team to develop the **Internship Logging & Evaluation System (ILES)**. The system manages the full internship cycle for undergraduate students. Key roles include **Student Interns**, **Workplace Supervisors**, **Academic Supervisors**, and **Internship Administrators**.

Each user interacts with a personalised dashboard:

- **Students** log weekly activities, submit logs, and view supervisor feedback.
- **Workplace Supervisors** review weekly logs and submit interim performance scores.
- **Academic Supervisors** approve logs, conduct visits, and submit final academic evaluations.
- **Internship Administrators** create placements, assign supervisors, and compute weighted total scores.

A weekly log moves through four states: **Draft → Submitted → Reviewed → Approved**. A student's total score is computed from three components — workplace performance (40%), academic evaluation (30%), and weekly logbook quality (30%).

The system uses **Django (backend)**, **React (frontend)**, and is deployed on **Heroku**. APIs are built with **Django REST Framework**. Real-time updates are sent via email and toast notifications. Use the case above to answer the following questions.

Question 1 — Models, ForeignKey and ORM (20 Marks)

In ILES, each weekly log is filed by a student under a specific internship placement. Both students and supervisors belong to departments.

a) Explain how `ForeignKey` relationships are used in Django to represent the following in the ILES system. (4 Marks)

- i. Each weekly log belongs to one internship placement.
- ii. Each student and supervisor belongs to a department.

b) The code below defines models for `Department`, `User`, and `WeeklyLog`. Lines 3, 7, and 12 are missing important details. Fill in the missing parts to complete the model relationships. (10 Marks)

```
1 | class Department(models.Model):
2 |     name = models.CharField(max_length=100)
3 |     # Add __str__ method here
4 | class User(AbstractUser):
5 |     is_student = models.BooleanField(default=False)
6 |     is_supervisor = models.BooleanField(default=False)
7 |     # Add relationship to Department
8 | class WeeklyLog(models.Model):
9 |     week_number = models.PositiveIntegerField()
10 |     activities = models.TextField()
11 |     status = models.CharField(max_length=20, default="Draft")
12 |     # Add relationship to placement
13 |     submitted_at = models.DateTimeField(auto_now_add=True)
```

c) Write a Django query to retrieve all weekly logs that are not yet Approved and were submitted by students in the same department as the currently logged-in academic supervisor (`request.user`). (6 Marks)

Model answer — Q1

a) A `ForeignKey` declares a many-to-one link from the model that owns the FK to the model named in its first argument. The owning row stores the parent's primary key.

i. A weekly log belongs to one placement, so `WeeklyLog` has `placement = models.ForeignKey(Placement, on_delete=models.CASCADE, related_name='logs')`. Many logs can share one placement; deleting the placement cascades and removes its logs.

ii. Both students and supervisors are `User` rows that share a `department = models.ForeignKey(Department, on_delete=models.SET_NULL, null=True, related_name='users')`. They share the same FK because both roles need it; `SET_NULL` keeps users alive if a department is deleted.

b)

```
1 | class Department(models.Model):
2 |     name = models.CharField(max_length=100)
3 |     def __str__(self): return self.name
4 | class User(AbstractUser):
5 |     is_student = models.BooleanField(default=False)
6 |     is_supervisor = models.BooleanField(default=False)
7 |     department = models.ForeignKey(Department, on_delete=models.SET_NULL, null=True, blank=True, related_name='users')
8 | class WeeklyLog(models.Model):
9 |     week_number = models.PositiveIntegerField()
10 |     activities = models.TextField()
11 |     status = models.CharField(max_length=20, default="Draft")
12 |     placement = models.ForeignKey(Placement, on_delete=models.CASCADE, related_name='logs')
13 |     submitted_at = models.DateTimeField(auto_now_add=True)
```

c)

```
WeeklyLog.objects.filter(
    placement__student__department=request.user.department
).exclude(status='Approved')
```

Question 2 — REST APIs and React data fetching (20 Marks)

As a frontend developer on ILES you are building the page where a student sees the weekly logs they have submitted. The data comes from a backend REST API.

a) Using an appropriate diagram, explain a REST API and describe how it helps frontend developers build features like the student log list in ILES. (4 Marks)

b) You have been provided with the following API: (6 Marks)

```
GET /api/student-logs/
Description: Returns a list of weekly logs submitted by the currently logged-in student.
Response Example:
[
  { "id": 1, "week_number": 1, "status": "Approved", "hours": 20 },
  { "id": 2, "week_number": 2, "status": "Submitted", "hours": 18 }
]
```

i. What HTTP method and URL should you use to call this API? **(2 marks)** ii. What is the expected response format? **(2 marks)** iii. How would you identify which logs are still pending (i.e. not yet Approved) in the frontend? **(2 marks)**

c) The following React component should fetch the list of student logs and display them in a table. Lines 2, 5, and 9 are incorrect or missing. Fix the missing parts to complete the component. (10 Marks)

```
1 | function StudentLogTable() {
2 |   const [logs, setLogs] = useEffect([]);
3 |
4 |   useEffect(() => {
5 |     fetch('_____')
6 |       .then(res => res.json())
7 |       .then(data => setLogs(data));
8 |   }, []);
9 |   return logs.map(log => (
10|     <tr>
11|       <td>{log.week_number}</td>
12|       <td>{log.status}</td>
13|       <td>{log.hours}</td>
14|     </tr>
15|   ));
16| }
```

Model answer — Q2

a) A REST API is an HTTP interface exposing resources by URL (e.g. `/api/student-logs/`) and operating on them with standard verbs (GET, POST, PUT, PATCH, DELETE). It returns structured JSON so any client — React, mobile, CLI — can read the same data without touching the database. For ILES, the React `StudentLogTable` component calls `GET /api/student-logs/` and renders the returned JSON; backend and frontend share no code and can evolve independently. (Sketch a diagram: React box ↔ HTTP/JSON arrow ↔ Django REST box ↔ Postgres box.)

b)

i. **GET**, URL `/api/student-logs/`. ii. **JSON** — specifically a JSON array of objects, each with `id`, `week_number`, `status`, `hours`. iii. Filter the array on the client: `const pending = logs.filter(l =>`

`l.status !== 'Approved');`, or render conditionally
`{log.status !== 'Approved' && <Badge>Pending</Badge>}`.

c)

```
1 | function StudentLogTable() {  
2 |   const [logs, setLogs] = useState([]); // useState, not useEffect  
3 |  
4 |   useEffect(() => {  
5 |     fetch('/api/student-logs/') // the URL from part (b)  
6 |       .then(res => res.json())  
7 |       .then(data => setLogs(data));  
8 |   }, []);  
9 |   return logs.map(log => ( // wrap in <table><tbody>  
if required; key is critical  
10 |     <tr key={log.id}>  
11 |       <td>{log.week_number}</td>  
12 |       <td>{log.status}</td>  
13 |       <td>{log.hours}</td>  
14 |     </tr>  
15 |   ));  
16 | }
```

Trap: line 2 swaps `useEffect` for `useState`. The third fix is adding `key={log.id}` on the mapped element so React can reconcile the list.

Question 3 — useState + controlled forms (20 Marks)

In ILES, students submit a weekly log using a form. The React component must handle inputs, send data to the backend, and feed back to the user.

a) Explain how `useState` and the `onChange` event handler are used together to manage form input in React. Why is this approach important when building interactive forms? (4 Marks)

b) Below is an incomplete React component. It should: track user input in the form fields (2 Marks); submit the form to `/api/logs/` (4 Marks); display a success message on submission (4 Marks). Lines 3, 6, and 11 are missing or incorrect. Fill them in. (10 Marks)

```
1 | function LogForm() {
2 |   const [formData, setFormData] = useState({ week_number: "", activities: "", hours:
  "" });
3 |   const [success, setSuccess] = useEffect();
4 |
5 |   const handleChange = (e) => {
6 |     _____
7 |   };
8 |
9 |   const handleSubmit = async (e) => {
10 |     e.preventDefault();
11 |     const res = await fetch('_____', {
12 |       method: "POST",
13 |       headers: { 'Content-Type': 'application/json' },
14 |       body: JSON.stringify(formData)
15 |     });
16 |     if (res.ok) {
17 |       setSuccess(true);
18 |     }
19 |   };
20 |
21 |   return (
22 |     <form onSubmit={handleSubmit}>
23 |       <input name="week_number" onChange={handleChange} value={formData.week_number} />
24 |       <textarea name="activities" onChange={handleChange} value={formData.activities} />
25 |       <input name="hours" onChange={handleChange} value={formData.hours} />
26 |       <button type="submit">Submit</button>
27 |       {success && <p>Log submitted successfully!</p>}
28 |     </form>
29 |   );
30 | }
```

c) Modify the form so a "Submitting..." message is shown while the request is in flight. Briefly describe what extra state you would add, where you would update it in `handleSubmit`, and how you would render it. (6 Marks)

Model answer — Q3

a) `useState` gives a component a value plus a setter function. Binding `<input value={formData.x} onChange={(e)=>setFormData(...)>` makes React own the input value (a "controlled" input). On every keystroke `onChange` fires, the state updates, and the component re-renders with the new value. This is important because it lets you validate as the user types, disable the Submit button until the form is valid, and reset the form after submission — none of which is possible with an uncontrolled input.

b)

```

1 | function LogForm() {
2 |   const [formData, setFormData] = useState({ week_number: "", activities: "", hours: "" });
3 |   const [success, setSuccess] = useState(false); // useState,
  with initial false
4 |
5 |   const handleChange = (e) => {
6 |     setFormData({ ...formData, [e.target.name]: e.target.value }); // generic
  controlled-form handler
7 |   };
8 |
9 |   const handleSubmit = async (e) => {
10 |    e.preventDefault();
11 |    const res = await fetch('/api/logs/',
  { // the correct endpoint
12 |      method: "POST",
13 |      headers: { 'Content-Type': 'application/json' },
14 |      body: JSON.stringify(formData)
15 |    });
16 |    if (res.ok) {
17 |      setSuccess(true);
18 |    }
19 |   };
  ...

```

c) Add a fourth piece of state: `const [submitting, setSubmitting] = useState(false);`. At the very start of `handleSubmit`, call `setSubmitting(true)`. After the fetch resolves (success or failure), call `setSubmitting(false)` — most safely inside a `finally` block. Render `{submitting && <p>Submitting...</p>}`, and add `disabled={submitting}` to the submit button so the user can't double-click.

```

const [submitting, setSubmitting] = useState(false);

const handleSubmit = async (e) => {
  e.preventDefault();
  setSubmitting(true);
  try {
    const res = await fetch('/api/logs/', { /* ... */ });
    if (res.ok) setSuccess(true);
  } finally {
    setSubmitting(false);
  }
};

// In the JSX:
<button type="submit" disabled={submitting}>
  {submitting ? 'Submitting...' : 'Submit'}
</button>
{submitting && <p>Submitting...</p>}

```

Question 4 — Django signals + React toast (20 Marks)

a) The following Django signal is intended to send an email when a weekly log's status changes to "Approved". However, the implementation has bugs in lines 2, 3, and 8. Correct only the marked lines so the signal works as expected, and only sends the email when an existing log is updated to "Approved". (12 Marks)

```
1 | @receiver(post_save, sender=WeeklyLog)
2 | def notify_on_approve(instance, **kwargs):
3 |     if instance.status = "Approved":
4 |         send_mail(
5 |             "Log Approved",
6 |             f"Your week {instance.week_number} log was approved.",
7 |             "admin@iles.com",
8 |             instance.placement.student.email
9 |         )
```

b) The React code below should display a toast when an issue update API response shows "Approved" as the status. There are bugs in lines 1, 4, and 5. Identify and fix only the incorrect lines so the toast displays correctly when the status is "Approved". (8 Marks)

```
1 | const updateLog = async () {
2 |     const res = await fetch('/api/logs/5/', { method: 'PATCH' });
3 |     const data = await res.json();
4 |     if (data.status = "Approved") {
5 |         Toast(res.status);
6 |     }
7 | };
```

Model answer — Q4

a)

```
1 | @receiver(post_save, sender=WeeklyLog)
2 | def notify_on_approve(sender, instance, created, **kwargs):           # signature must
   accept sender, instance, created
3 |     if not created and instance.status == "Approved":                # `not created`
   ensures it's an update, == not =
4 |         send_mail(
5 |             "Log Approved",
6 |             f"Your week {instance.week_number} log was approved.",
7 |             "admin@iles.com",
8 |             [instance.placement.student.email]                        # recipient_list
   must be a LIST
9 |         )
```

Mark allocation: signature fix (4), not created + == (4), wrapping the recipient in a list (4).

b)

```

1 | const updateLog = async () => {                                // arrow function
  needs `=>`
2 |   const res = await fetch('/api/logs/5/', { method: 'PATCH' });
3 |   const data = await res.json();
4 |   if (data.status === "Approved") {                            // === not =
5 |     Toast("Approved!");                                        // model status
  string, not HTTP res.status
6 |   }
7 | };

```

Question 5 — APIView + comments on each line (20 Marks)

a) Describe the role of an API in the ILES project, focusing on how it enables interaction between the React frontend and the Django backend. (4 Marks)

b) You are building an endpoint for ILES where students view the list of weekly logs they have submitted. Complete the missing lines to create a Django REST Framework view that handles a GET request to `/api/student-logs/`, returning logs for the currently logged-in student. The code has gaps in lines 3, 5, and 7. Fill them correctly and add a comment on what your code does (format: `Line 3 | modified code # comment`). (6 Marks)

```

1 | from rest_framework.views import APIView
2 | from rest_framework.response import Response
3 | from .models import _____
4 | from .serializers import WeeklyLogSerializer
5 | class StudentLogsView(_____):
6 |     def get(self, request):
7 |         logs = WeeklyLog.objects.filter(_____)
8 |         serializer = WeeklyLogSerializer(logs, many=True)
9 |         return Response(serializer.data)

```

c) You are now implementing the React frontend to fetch the student's logs and display them in an unordered list `<ul>`. The code below has mistakes in lines 2, 5, and 10. Correct them and add a comment on what your code does. (10 Marks)

```

1 | function LogDashboard() {
2 |   const [logs, setLogs] = useEffect([]);
3 |
4 |   useEffect(() => {
5 |     fetch('/api/_____')
6 |       .then(response => response)
7 |       .then(data => setLogs(data));
8 |   }, []);
9 |
10|   {logs.map(log => (
11|     <li key={log.id}>Week {log.week_number} - {log.status}</li>
12|   ))}
13| }

```

Model answer — Q5

a) The API is the contract between the React frontend and the Django backend. Django exposes endpoints like `GET /api/student-logs/` and `POST /api/logs/` that emit/accept JSON. React calls those endpoints with `fetch`, receives the JSON, and updates its UI state. Without an API, the frontend would have to share a database or in-process code with Django, breaking the separation between presentation and business logic. In ILES, every action a student takes — submit a log, view feedback, see status — flows through one of these endpoints.

b)

```
Line 3 | from .models import WeeklyLog
# import the model whose objects we will return
Line 5 | class StudentLogsView(APIView):           # subclass APIView so we can define
get() ourselves
Line 7 | logs = WeeklyLog.objects.filter(placement__student=request.user) # scope to
current student
```

c)

```
Line 2 | const [logs, setLogs] = useState([]);           // useState, not useEffect;
default []
Line 5 | fetch('/api/student-logs/')                       // correct endpoint
Line 10| return (<ul>{logs.map(log =>
(                                     // returned JSX must be wrapped in <ul>
```

(Note: depending on how the question reads, the line 10 fix could also be on line 6 where `response` should be `response.json()`. Both are valid bug points — mention them.)

Complete corrected component:

```
function LogDashboard() {
  const [logs, setLogs] = useState([]);           // state to hold fetched logs

  useEffect(() => {
    fetch('/api/student-logs/')                   // call API endpoint
      .then(response => response.json())           // parse body
      .then(data => setLogs(data));               // store in state
  }, []);                                         // run once on mount

  return (
    <ul>
      {logs.map(log => (
        <li key={log.id}>Week {log.week_number} - {log.status}</li>
      ))}
    </ul>
  );
}
```

Question 6 — Heroku deployment (20 Marks)

You are deploying ILES to Heroku. Correct configuration is critical for security and stability.

a) List two important settings that must be configured in Django when deploying to Heroku and explain why each is necessary. (4 Marks)

b) The following Django settings snippet is incomplete or incorrect. Carefully review the numbered lines and fix the mistakes in lines 2, 4, and 7. (8 Marks)

```
1 | import os
2 | DEBUG = True
3 |
4 | ALLOWED_HOSTS = []
5 |
6 | DATABASES = {
7 |     'default': { 'ENGINE': 'django.db.backends.sqlite3', 'NAME': BASE_DIR /
'db.sqlite3' }
8 | }
9 |
10| STATIC_ROOT = ''
```

c) Below are deployment files used to deploy ILES on Heroku. There are mistakes in the Procfile (line 2) and environment variables setup (line 5). Fix the mistakes. (8 Marks)

```
Procfile
1 | web: gunicorn iles.wsgi
2 | (missing settings here)

.env File (Environment Variables)
3 | SECRET_KEY=mysecretkey
4 | DEBUG=True
5 | ALLOWED_HOSTS=*
```

Model answer — Q6

a) Two critical settings:

1. **DEBUG = False** — keeps stack traces, SQL queries and the SECRET_KEY out of production error pages.
2. **ALLOWED_HOSTS** — must list the Heroku app's hostname (e.g. `iles-app.herokuapp.com`); otherwise Django rejects every request as a host-header attack.

Other valid pairs: **SECRET_KEY** from env, **DATABASE_URL** -backed Postgres, **STATIC_ROOT** + WhiteNoise.

b)

```
1 | import os
2 | DEBUG = False                                # never
True in production
3 |
4 | ALLOWED_HOSTS = ['iles-app.herokuapp.com', '.herokuapp.com'] # or read
from env
5 |
6 | DATABASES = {
7 |     'default': dj_database_url.config(default=os.environ.get('DATABASE_URL')) # Heroku
uses Postgres, not SQLite
8 | }
9 |
10| STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
# required for collectstatic
```

c)

```
Procfile
1 | web: gunicorn iles.wsgi --log-file -
2 | release: python manage.py migrate --noinput          # missing release phase added

.env File
3 | SECRET_KEY=mysecretkey                             # OK (rotate before pushing live)
4 | DEBUG=False                                         # never True in prod
5 | ALLOWED_HOSTS=iles-app.herokuapp.com                # never '*' in prod
```

Question 7 — Serializer validation + frontend error handling (20 Marks)

In ILES, students submit weekly logs through a form. The backend must validate the data and the frontend should display friendly error messages when validation fails.

a) Explain the importance of input validation in backend APIs and why error feedback matters in frontend UI. (4 Marks)

b) Below is a Django REST Framework serializer for the WeeklyLog model. It should ensure that both `week_number` and `activities` are required, and that `hours` is between 0 and 80. There are missing lines in 3, 5, and 6. Fill in the missing code to implement proper validation. (8 Marks)

```
1 | from rest_framework import serializers
2 | from .models import WeeklyLog
3 | class ____ (serializers.ModelSerializer):
4 |     class Meta:
5 |         model = ____
6 |         fields = ____
7 |
8 |     def validate(self, data):
9 |         if not data.get('week_number') or not data.get('activities'):
10 |             raise serializers.ValidationError("Week number and activities are required.")
11 |         if data.get('hours', 0) < 0 or data.get('hours', 0) > 80:
12 |             raise serializers.ValidationError("Hours must be between 0 and 80.")
13 |         return data
```

c) Below is a React form for submitting logs. It works on success but fails silently when the server returns an error. Fix lines 3, 5, and 8 to: catch API errors, store the error message, and display it below the form. (8 Marks)

```
1 | function LogForm() {
2 |   const [formData, setFormData] = useState({ week_number: "", activities: "", hours:
   "" });
3 |   const [error, setError] = useEffect();
4 |
5 |   const handleSubmit = async (e) {
6 |     e.preventDefault();
7 |     const res = await fetch('/api/logs/', {
8 |       method: POST,
9 |       headers: { 'Content-Type': 'application/json' },
10 |      body: JSON.stringify(formData)
11 |    });
12 |    if (!res.ok) {
13 |      // handle error
14 |    }
15 |  };
16 |
17 |  return (
18 |    <form onSubmit={handleSubmit}>
19 |      <input name="week_number" value={formData.week_number} />
20 |      <textarea name="activities" value={formData.activities} />
21 |      <input name="hours" value={formData.hours} />
22 |      <button type="submit">Submit</button>
23 |      {error && <p className="error">{error}</p>}
24 |    </form>
25 |  );
26 | }
```

Model answer — Q7

a) Validation in the backend is the only authoritative line of defence — frontends can be bypassed (curl, Postman, malicious clients). Centralising validation in DRF serializers protects the database from bad types, missing required fields, oversized payloads and injection attempts, and produces consistent 400-level errors regardless of client. Surfacing those errors in the frontend matters because silent failures look like the app is broken; clear messages tell the user exactly what to fix ("Hours must be between 0 and 80") and reduce support load.

b)

```
3 | class WeeklyLogSerializer(serializers.ModelSerializer):
4 |     class Meta:
5 |         model = WeeklyLog
6 |         fields = ['id', 'week_number', 'activities', 'hours', 'status'] # or '__all__'
```

c)

```

3 |   const [error, setError] = useState(null);                                // useState,
   default null
5 |   const handleSubmit = async (e) => {                                     // arrow needs
   `=>`
8 |       method:
   "POST",                                                                    // POST must be a string
...
12|   if (!res.ok) {
13|       const errBody = await res.json();
14|       setError(errBody.detail || JSON.stringify(errBody));
15|   }

```

Full corrected `handleSubmit`:

```

const handleSubmit = async (e) => {
  e.preventDefault();
  setError(null);
  const res = await fetch('/api/logs/', {
    method: "POST",
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(formData),
  });
  if (!res.ok) {
    const errBody = await res.json();
    setError(errBody.detail || JSON.stringify(errBody));
  }
};

```

Question 8 — Django testing (20 Marks)

The ILES system must be thoroughly tested. You are asked to implement and fix Django unit tests for the WeeklyLog model and API.

a) Explain why automated testing is important in backend development, and give one reason a test might fail even when the code seems correct. (4 Marks)

b) Write a Django test case method that checks that a logged-in student can successfully create a weekly log through the API. The test must: (8 Marks)

- Create a test user (student),
- Log the user in,
- Submit a `POST` request to `/api/logs/` with valid log data,
- Assert that the response status code is `201`.

c) The test below is meant to check that a workplace supervisor cannot view all logs across the whole university, but it is failing. Review the numbered lines and fix the errors in lines 2, 5, and 6. (8 Marks)

```

1 | class LogAccessTest(TestCase):
2 |     def view_all_logs(self):
3 |         supervisor = User.objects.create(username="bob", password="pass123")
4 |         self.client.login(username="bob", password="pass123")
5 |         response = self.client.post('/api/logs/')
6 |         self.assertEqual(response.status_code, 200)

```

Model answer — Q8

a) Automated tests catch regressions, document expected behaviour, and let you refactor without fear. They can fail despite code that "looks correct" because of hidden assumptions — for example, the test forgot to log the user in and the endpoint silently returns a 401, or the database is empty so a queryset is empty when the test assumed otherwise, or a timezone difference makes a date assertion off by a day.

b)

```
from rest_framework.test import APITestCase
from rest_framework import status
from django.contrib.auth import get_user_model

User = get_user_model()

class WeeklyLogCreateTest(APITestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username='alice', password='pass123', role='student'
        )
        self.client.login(username='alice', password='pass123')

    def test_student_can_create_log(self):
        data = {
            'week_number': 1,
            'activities': 'Wrote technical specs',
            'hours': 20,
        }
        response = self.client.post('/api/logs/', data, format='json')
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
```

c)

```
1 | class LogAccessTest(TestCase):
2 |     def test_supervisor_cannot_view_all_logs(self):           # test_*
    prefix required
3 |         supervisor = User.objects.create_user(username="bob", password="pass123") #
    create_user, not create
4 |         self.client.login(username="bob", password="pass123")
5 |         response = self.client.get('/api/logs/')              # GET to
    read, not POST
6 |         self.assertEqual(response.status_code, 403)          # 403
    Forbidden, not 200 OK
```

Question 9 — Weighted score computation (20 Marks) (ILES-specific, expect this)

The ILES internship administrator computes a student's total score from three components: **Workplace Performance (40%)**, **Academic Evaluation (30%)** and **Logbook Quality (30%)**.

a) Explain why weighted scoring is preferred over a simple average, and one risk you would mitigate when implementing it. (4 Marks)

b) Complete the `Evaluation` model so that `total` is automatically computed and stored whenever the record is saved. Fields are partially given — fill in lines 5, 9, and 11. (10 Marks)

```
1 | from decimal import Decimal
2 | from django.db import models
3 |
4 | class Evaluation(models.Model):
5 |     student = models.ForeignKey(_____, on_delete=models.CASCADE,
   |     related_name='evaluations')
6 |     workplace_score = models.DecimalField(max_digits=5, decimal_places=2)
7 |     academic_score = models.DecimalField(max_digits=5, decimal_places=2)
8 |     logbook_score = models.DecimalField(max_digits=5, decimal_places=2)
9 |     total = _____
10|
11| def save(self, *args, **kwargs):
12|     self.total = _____
13|     super().save(*args, **kwargs)
```

c) Write a serializer-level check that prevents a second `Evaluation` from being saved for the same student. (6 Marks)

Model answer — Q9

a) Weighted scoring lets the institution emphasise components that matter most — e.g. real workplace performance dominates over self-reported logbook entries — instead of treating every component as equal. The risk to mitigate is **mixing types**: multiplying a Django `DecimalField` by a Python `float` raises `TypeError` and corrupts totals; always cast weights with `Decimal('0.4')`. Other valid risks: forgetting to clamp scores to 0–100, double-counting if `save()` is called twice without resetting `total`, or weights that don't sum to 1.

b)

```
5 | student = models.ForeignKey(CustomUser, on_delete=models.CASCADE,
   | related_name='evaluations')
9 | total = models.DecimalField(max_digits=5, decimal_places=2, blank=True, null=True)
12| self.total = (
   |     self.workplace_score * Decimal('0.4') +
   |     self.academic_score * Decimal('0.3') +
   |     self.logbook_score * Decimal('0.3')
   | )
```

c)

```

class EvaluationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Evaluation
        fields = '__all__'

    def validate(self, data):
        student = data.get('student')
        # On create only – skip when updating an existing instance
        if self.instance is None and Evaluation.objects.filter(student=student).exists():
            raise serializers.ValidationError(
                "An evaluation already exists for this student."
            )
        return data

```

Alternative (model-level): `class Meta: unique_together = ('student',)` on the `Evaluation` model.

Question 10 — Workflow state transitions (20 Marks) (ILES-specific, expect this)

A weekly log moves through **Draft → Submitted → Reviewed → Approved**. Invalid transitions (e.g. Draft → Approved) must be rejected. The system must also record who made each transition.

a) Explain what a state machine is and why it is a good fit for the weekly log review workflow. (4 Marks)

b) Complete the `WeeklyLog.save()` override so an invalid transition raises a `ValidationError`. Fill lines 5, 8, and 11. (10 Marks)

```

1 | from django.core.exceptions import ValidationError
2 |
3 | class WeeklyLog(models.Model):
4 |     # ... fields ...
5 |     ALLOWED = _____
6 |
7 |     def save(self, *args, **kwargs):
8 |         if _____: # only check on updates
9 |             old = WeeklyLog.objects.get(pk=self.pk)
10 |             if old.status != self.status:
11 |                 if _____:
12 |                     raise ValidationError(
13 |                         f"Cannot move from {old.status} to {self.status}"
14 |                     )
15 |             super().save(*args, **kwargs)

```

c) Write a small `StatusHistory` model that records every status change, with FK to the log, the old status, the new status, the user who changed it, and a timestamp. (6 Marks)

Model answer — Q10

a) A state machine is a model with a finite set of states and explicit rules for which transitions are allowed between them. For weekly logs it's an ideal fit because the document has a clear lifecycle: only the student can submit a Draft, only a supervisor can review a Submitted log, only an academic supervisor can approve a Reviewed log, and a log should never skip stages (e.g. Draft → Approved).

Encoding the allowed transitions in code prevents inconsistent data and removes the need to scatter `if` statements across views.

b)

```
5 |     ALLOWED = {
      'Draft':      ['Submitted'],
      'Submitted':  ['Reviewed', 'Draft'],
      'Reviewed':   ['Approved', 'Submitted'],
      'Approved':   [],
    }

...
8 |     if self.pk:                                     # truthy only on existing
rows
...
11|         if self.status not in self.ALLOWED[old.status]:
```

c)

```
class StatusHistory(models.Model):
    log = models.ForeignKey(
        WeeklyLog, on_delete=models.CASCADE, related_name='history')
    old_status = models.CharField(max_length=20)
    new_status = models.CharField(max_length=20)
    changed_by = models.ForeignKey(
        CustomUser, on_delete=models.SET_NULL, null=True,
        related_name='log_changes')
    changed_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.log_id}: {self.old_status} -> {self.new_status}"
```

Appendix A — Bug catalogue (memorise these traps)

These are the bugs the AITS paper actually planted, and the ones most likely to be reused in ILES:

#	Trap	Right form
1	<code>useState([])</code> written as <code>useEffect([])</code> on the state line	<code>const [x, setX] = useState([])</code>
2	<code>if (x = 'val')</code> (assignment) in JS	<code>if (x === 'val')</code> (strict equality)
3	<code>if (status = 'Approved'):</code> in Python	<code>if status == 'Approved':</code>
4	<code>recipient_list</code> passed as a string	wrap in <code>[...]</code>
5	Signal handler missing <code>created</code> arg	<code>def h(sender, instance, created, **kwargs)</code>
6	Signal fires on create as well as update	<code>if not created and ...</code>
7	<code>User.objects.create(...)</code> in tests, then <code>login()</code> fails	use <code>create_user(...)</code> so password is hashed
8	Test method named <code>view_all_logs</code>	must start with <code>test_</code>
9	<code>self.client.post()</code> when you meant GET	match the verb to the action
10	<code>DEBUG = True</code> in prod	<code>DEBUG = False</code> , drive via env var
11	<code>ALLOWED_HOSTS = []</code> with <code>DEBUG False</code>	list your real hostname(s)
12	<code>ALLOWED_HOSTS=*</code> in prod env	never wildcard in production
13	<code>STATIC_ROOT = ''</code>	<code>os.path.join(BASE_DIR, 'staticfiles')</code>
14	SQLite in Heroku settings	use <code>dj_database_url</code> with <code>DATABASE_URL</code>
15	<code>Toast(res.status)</code> (HTTP code)	<code>Toast(data.status)</code> (model field)
16	Missing <code>await res.json()</code>	always await body before comparing
17	Forgotten <code>key={...}</code> in <code>.map(...)</code>	every mapped element needs a stable key
18	<code>method: POST</code> (bare identifier)	<code>method: "POST"</code> (string)
19	<code>async (e) {}</code> without <code>=></code>	<code>async (e) => {}</code>
20	<code>response => response</code> instead of <code>response.json()</code>	parse JSON body
21	<code>Decimal * float</code> raises <code>TypeError</code>	use <code>Decimal('0.4')</code>
22	Forgetting <code>serializer.is_valid()</code> before <code>.save()</code>	always validate first
23	<code>permission_classes = IsAuthenticated</code> (not a list)	<code>[IsAuthenticated]</code>
24	<code>fields = id, name</code> (tuple in <code>Meta</code>)	<code>fields = ['id', 'name']</code>
25	<code>class Meta</code> declared outside the serializer	must be nested inside the class

Appendix B — Five 4-mark concept paragraphs to memorise

You will get **at least three** concept questions for 4 marks each. These are pre-written; you only need to recall the structure.

"Explain a REST API"

A REST API is an HTTP interface exposing resources by URL and operating on them with the standard verbs GET, POST, PUT, PATCH, DELETE. It returns structured data — almost always JSON — so any client (React, mobile, CLI) can use the same endpoints without sharing code or a database. The separation lets frontend and backend evolve independently. In ILES the React UI calls `/api/student-logs/` to render the student's logs.

"Explain useState + onChange in React forms"

`useState` gives a component a value plus a setter. Binding an input's `value` to that state and its `onChange` to the setter makes it a controlled input: React owns the value, the component re-renders on every keystroke, and the form is validated before submission. This is essential for interactive forms — without it you cannot validate live, disable Submit conditionally, or reset the form.

"Why input validation in backend APIs"

Validation in the backend is the only authoritative defence because frontends can be bypassed. DRF serializers reject bad types, missing required fields, and out-of-range numbers, returning 400-level errors with messages. Surfacing those errors in the frontend (e.g. "Hours must be between 0 and 80") tells the user exactly what to fix instead of failing silently.

"Why automated testing"

Automated tests catch regressions, document expected behaviour, and let you refactor safely. They can fail despite code that "looks correct" — for example, the test forgot to log the user in, or the database is empty so a queryset is empty when the test expected data. In ILES, tests should cover the four state transitions, RBAC, and the scoring formula.

"Role of the API in the Django/React split"

Django handles authentication, persistence and business logic (state transitions, weighted scoring) and exposes that through REST endpoints. React owns the UI: it fetches JSON, renders dashboards and forms, and sends user actions back as HTTP requests. Neither side imports the other; they communicate only over HTTP. The API is the contract that allows the two halves to evolve independently.

Appendix C — One-page exam-day plan

1. Spend 5 minutes reading every question. **Pick your 5.**
2. **Start with your strongest** — bank marks early, build confidence.
3. For every question, write the 4-mark concept paragraph FIRST (easy marks), then the code.
4. **Always write imports**, even for fill-ins. Markers reward them.
5. After 30 minutes on one question, **move on**. Come back if time allows.
6. Last 10 minutes: re-read your code. Check `==` vs `=`, missing colons, missing brackets, `useState` vs `useEffect`, `key={...}` in maps.
7. If you have time, write `# explanation` comments — Q5 last year demanded them; offering them unprompted impresses the marker.

Question 11 — SDLC + Requirements engineering (20 Marks) (Lecture 1 + 2)

A development team is starting work on ILES. Before writing any code, the team must clarify what the system must do, who uses it, and how the build will proceed.

- a) List the six phases of the Software Development Life Cycle (SDLC) and give a one-line ILES-specific example of what happens in each phase. (6 Marks)
- b) Write two user stories for the Workplace Supervisor role in the prescribed format **As a [role], I want to [action], so that [benefit]**. (4 Marks)
- c) For each of the two user stories you wrote, derive one functional requirement in the form "The system shall ...". (4 Marks)
- d) Distinguish between functional and non-functional requirements, and give three non-functional requirements relevant to ILES (one of which must be measurable). (6 Marks)

Model answer — Q11

a) SDLC phases applied to ILES:

Phase	ILES example
Planning	Decide scope (only Makerere internships), tech stack (Django + React + Postgres), team roles.
Analysis	Collect user stories from each of the four roles, identify entities and workflow states.
Design	Draw the ERD (CustomUser-Placement-WeeklyLog-Evaluation), define API endpoints, sketch dashboards.
Implementation	Code Django models, DRF serializers/views, React forms and dashboards.
Testing	Unit tests for state transitions and scoring; integration tests for the auth flow.
Deployment	Push to Heroku with gunicorn + Postgres add-on, set env vars, run <code>collectstatic</code> and <code>migrate</code> .

b) Two workplace-supervisor stories:

1. As a workplace supervisor, I want to approve or reject submitted weekly logs so that only valid work is recorded.
2. As a workplace supervisor, I want to comment on a student's weekly log so that they can improve their reporting.

c) Functional requirements:

1. The system shall allow a workplace supervisor to update the status of a `WeeklyLog` from `Submitted` to `Reviewed` or `Rejected`.
2. The system shall allow a workplace supervisor to attach a free-text comment to a weekly log; the comment shall be stored with a timestamp and the author's ID.

d) Functional requirements describe **what the system does** (verb-led, "shall ..."); non-functional requirements describe **how it behaves** (often measurable adjectives — performance, security, scalability).

Three NFRs for ILES (third is measurable):

1. **Security** — only assigned supervisors can read or modify a student's logs.
2. **Auditability** — every status change must be recorded with the user who made it and the timestamp.
3. **Performance** — the student dashboard must load within 3 seconds with 100 logs in the database (measurable).

Question 12 — System architecture + request flow (20 Marks) (Lecture 3)

a) Using a labelled diagram, describe the four-tier architecture of ILES and explain the responsibility of each tier. (6 Marks)

b) List the five steps that occur when a student submits a weekly log, from the moment they click "Submit" in React to the moment the dashboard updates. (6 Marks)

c) Take the user story "As a student, I want to submit a weekly log so that my supervisor can review it" and extract: (i) the entities implied by the story, (ii) the attributes the WeeklyLog entity should have to support it, (iii) the relationships between the entities. (8 Marks)

Model answer — Q12

a) Four-tier architecture:



- **React Frontend** owns the UI: rendering, controlled forms, client-side validation, calling `fetch`.
- **Django REST API** exposes resources by URL and verb (`GET /api/logs/`); serializers map between JSON and model instances.
- **Django Backend** is the business-logic centre: models, state-transition rules, signals, weighted-score computation.
- **PostgreSQL** persists all data. Django talks to it through the ORM.

b) Five-step request flow:

1. **React form submission** — `e.preventDefault()`, `fetch('/api/logs/', { method: 'POST', body: JSON.stringify(formData) })`.
2. **API request** — HTTP POST hits the URL routed in `urls.py` to the matching view.
3. **Serializer validation** — DRF's `WeeklyLogSerializer(data=request.data).is_valid()` runs field validators, `validate_<field>`, and cross-field `validate()`.
4. **Database storage** — `serializer.save()` triggers `model.save()` (which can run state-transition checks) and writes the row via the ORM.
5. **JSON response** — Django returns `Response(serializer.data, status=201)`; React reads it, calls `setLogs(...)`, and the dashboard re-renders.

c)

- (i) **Entities** (nouns in the story): `Student` (i.e. `CustomUser` with role=`Student`), `WeeklyLog`, `Supervisor` (`CustomUser` with role=`WorkplaceSupervisor` or `AcademicSupervisor`). An implicit fourth entity, `InternshipPlacement`, ties the student to the supervisor.
- (ii) **Attributes** for `WeeklyLog`: `id` (PK), `week_number`, `activities` (text), `hours` (integer), `status` (choice from `Draft/Submitted/Reviewed/Approved/Rejected`), `submitted_at` (auto timestamp), `created_at`, `updated_at`.
- (iii) **Relationships**:
 - `WeeklyLog` → `InternshipPlacement`: many-to-one (`ForeignKey`, `related_name='logs'`).
 - `InternshipPlacement` → `CustomUser` (student): many-to-one.
 - `InternshipPlacement` → `CustomUser` (supervisor): many-to-one (a supervisor may oversee many placements).
- Therefore `WeeklyLog` → `Student` is a **transitive many-to-one** via `placement`, which is why the standard query is `WeeklyLog.objects.filter(placement__student=request.user)`.

Question 13 — Authentication & Authorisation (20 Marks)

(Lecture 4)

ILES requires that only authenticated users access the system and that each role sees only what their permissions allow.

a) Distinguish between authentication and authorisation, and give one example of each in ILES. (4 Marks)

b) Briefly describe four authorisation mechanisms (RBAC, ABAC, DAC, MAC) and state which one ILES uses and why. (6 Marks)

c) The view below restricts approving a weekly log to users in the `AcademicSupervisor` group. Fill in the missing lines 3, 5, and 8 so the view (i) requires the user to be logged in, (ii) requires the `logs.can_approve_log` permission, and (iii) verifies the user is in the right group before approving. (6 Marks)

```
1 | from django.contrib.auth.decorators import _____, _____
2 | from django.shortcuts import get_object_or_404, redirect
3 | _____ # decorator: must be logged in
4 | _____ # decorator: must have
   logs.can_approve_log permission
5 | def approve_log(request, pk):
6 |     log = get_object_or_404(WeeklyLog, pk=pk)
7 |     if not _____: # group check
8 |         return redirect('forbidden')
9 |     log.status = 'Approved'
10 |     log.save()
11 |     return redirect('dashboard')
```

d) Write the React Router setup so a `/admin` route is only reachable by users whose role (stored in `localStorage`) equals `"admin"`, redirecting everyone else to `/login`. Use a `ProtectedRoute` wrapper. (4 Marks)

Model answer — Q13

a) Authentication verifies who a user is (e.g. a student logs in with username + password and Django checks the hashed credential). **Authorisation** decides what they can do once known (e.g. only users in the `AcademicSupervisor` group can call the "Approve log" endpoint).

b)

- **RBAC** — Role-Based Access Control. Permissions attach to roles (Student, Supervisor, Admin); users inherit role permissions. Used by ILES.
- **ABAC** — Attribute-Based Access Control. Permissions depend on attributes (department, time of day, device).
- **DAC** — Discretionary Access Control. The resource owner chooses who else may access it.
- **MAC** — Mandatory Access Control. Strict, centrally-enforced policies (used in military/government systems).

ILES uses **RBAC** because permissions are stable per role and don't depend on context like time or location, which makes the rules easy to enforce uniformly across views, dashboards, and the API.

c)

```
1 | from django.contrib.auth.decorators import login_required, permission_required
2 | from django.shortcuts import get_object_or_404, redirect
3 | @login_required                                     # logged-in only
4 | @permission_required('logs.can_approve_log', raise_exception=True) # permission check
5 | def approve_log(request, pk):
6 |     log = get_object_or_404(WeeklyLog, pk=pk)
7 |     if not request.user.groups.filter(name='AcademicSupervisor').exists():
8 |         return redirect('forbidden')
9 |     log.status = 'Approved'
10 |    log.save()
11 |    return redirect('dashboard')
```

d)

```
// ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
function ProtectedRoute({ allowedRole, children }) {
    const role = localStorage.getItem('role');
    if (!role) return <Navigate to="/login" replace />;
    if (allowedRole && role !== allowedRole)
        return <Navigate to="/login" replace />;
    return children;
}

// App.jsx
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/admin" element={
    <ProtectedRoute allowedRole="admin"><AdminDashboard /></ProtectedRoute>
  } />
</Routes>
```

Question 14 — Email, SMS, and Toast notifications (20 Marks) (Lecture 7)

ILES notifies students and supervisors when a weekly log changes state. The backend sends emails (and optionally SMS via Twilio); the frontend shows toasts.

a) Briefly explain why notifications matter in ILES and list the three types of notification covered in lecture 7. (4 Marks)

b) The Django `settings.py` snippet below is meant to configure Gmail SMTP. Lines 2, 4, and 6 are wrong or missing. Fix them. (6 Marks)

```
1 | EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
2 | EMAIL_HOST = 'gmail.com'
3 | EMAIL_PORT = 587
4 | EMAIL_USE_TLS = False
5 | EMAIL_HOST_USER = 'iles-noreply@gmail.com'
6 | EMAIL_HOST_PASSWORD = 'mygmailpassword'
```

c) Complete the Twilio helper function. Lines 4, 6, and 8 are missing. (6 Marks)

```
1 | from twilio.rest import Client
2 | from django.conf import settings
3 | def send_sms(to_number, body):
4 |     client = _____
5 |     message = client.messages.create(
6 |         _____=body,
7 |         from_=settings.TWILIO_PHONE_NUMBER,
8 |         _____=to_number
9 |     )
10|     return message.sid
```

d) The React component below should show a success toast on save. Fix lines 1, 5, and 7. (4 Marks)

```
1 | import Toastify from 'react-toastify';
2 | import 'react-toastify/dist/ReactToastify.css';
3 |
4 | function SaveButton() {
5 |     const onSave = () {
6 |         fetch('/api/logs/1/', { method: 'PATCH' })
7 |             .then(res => Toastify.success("Saved!"));
8 |     };
9 |     return <button onClick={onSave}>Save</button>;
10| }
```

Model answer — Q14

a) Notifications keep users informed of state changes that they would otherwise have to poll for — e.g. a student learning that their log was approved, a supervisor learning that a new log is awaiting review. This improves transparency, reduces support load, and tightens the workflow loop. The three types covered are **email notifications**, **SMS notifications**, and **in-app/visual (toast) notifications**.

b)

```
1 | EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
2 | EMAIL_HOST = 'smtp.gmail.com' # smtp.gmail.com, not gmail.com
3 | EMAIL_PORT = 587
4 | EMAIL_USE_TLS = True # TLS must be True for port 587
5 | EMAIL_HOST_USER = 'iles-noreply@gmail.com'
6 | EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_PASSWORD') # never hardcode credentials
```

c)

```
4 | client = Client(settings.TWILIO_ACCOUNT_SID, settings.TWILIO_AUTH_TOKEN)
6 |     body=body,
8 |     to=to_number
```

Complete:

```
def send_sms(to_number, body):
    client = Client(settings.TWILIO_ACCOUNT_SID, settings.TWILIO_AUTH_TOKEN)
    message = client.messages.create(
        body=body,
        from_=settings.TWILIO_PHONE_NUMBER,
        to=to_number,
    )
    return message.sid
```

d)

```
1 | import { toast } from 'react-toastify';           // named import
   { toast }
...
5 | const onSave = () =>                               // arrow function needs =>
   {
...
7 |     .then(res =>
   toast.success("Saved!"));                          // lowercase toast, not Toastify
```

You also need `<ToastContainer />` mounted once in `App` for any of this to render — credit if mentioned.

Question 15 — Admin features: filtering + assignment (20 Marks) (Lecture 6)

The Internship Administrator dashboard lists all weekly logs across the institution. The admin must be able to filter by status and reassign a log's reviewer.

a) Explain why filtering must happen on the backend with Django QuerySets rather than purely on the frontend. (4 Marks)

b) Complete the DRF view below so that `GET /api/admin/logs/` accepts a `?status=Submitted` query parameter and a `?department=<id>` query parameter, applying each only if present. Fill lines 4, 6, and 8. (8 Marks)

```
1 | from rest_framework.views import APIView
2 | from rest_framework.response import Response
3 | from .models import WeeklyLog
4 | from .serializers import _____
5 | class AdminLogListView(APIView):
6 |     permission_classes = [_____]
7 |     def get(self, request):
8 |         qs = WeeklyLog.objects._____
9 |         status_param = request.query_params.get('status')
10 |         if status_param:
11 |             qs = qs.filter(status=status_param)
12 |         dept_param = request.query_params.get('department')
13 |         if dept_param:
14 |             qs = qs.filter(placement__student__department_id=dept_param)
15 |         serializer = WeeklyLogSerializer(qs, many=True)
16 |         return Response(serializer.data)
```

c) Write a small Django REST view that handles `PATCH /api/logs/<id>/assign/` and sets the log's `assigned_to` field to the user ID supplied in the request body. Use any reasonable view style and include a permission check that only the `Admin` group can call it. (8 Marks)

Model answer — Q15

a) Frontend filtering only works if you've already shipped every row to the browser, which is impossibly expensive once the institution has thousands of logs. Backend filtering uses indexed SQL `WHERE` clauses through Django QuerySets, so only the needed rows cross the network. It also enforces security: the backend can hide rows the user is not allowed to see, whereas a frontend filter could be bypassed by anyone reading the JSON in DevTools.

b)

```
4 | from .serializers import WeeklyLogSerializer
6 | permission_classes = [IsAdminUser]           # or a custom IsAdmin group
permission
8 |     qs = WeeklyLog.objects.all()             # start with everything; filters
narrow it
```

Complete view:

```

from rest_framework.permissions import IsAdminUser

class AdminLogListView(APIView):
    permission_classes = [IsAdminUser]

    def get(self, request):
        qs = WeeklyLog.objects.all()
        status_param = request.query_params.get('status')
        if status_param:
            qs = qs.filter(status=status_param)
        dept_param = request.query_params.get('department')
        if dept_param:
            qs = qs.filter(placement__student__department_id=dept_param)
        serializer = WeeklyLogSerializer(qs, many=True)
        return Response(serializer.data)

```

c)

```

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticated, BasePermission
from django.shortcuts import get_object_or_404
from django.contrib.auth import get_user_model

User = get_user_model()

class IsAdminGroup(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and \
            request.user.groups.filter(name='Admin').exists()

class AssignLogView(APIView):
    permission_classes = [IsAuthenticated, IsAdminGroup]

    def patch(self, request, pk):
        log = get_object_or_404(WeeklyLog, pk=pk)
        assignee_id = request.data.get('assigned_to')
        if not assignee_id:
            return Response(
                {'detail': 'assigned_to is required.'},
                status=status.HTTP_400_BAD_REQUEST,
            )
        log.assigned_to = get_object_or_404(User, pk=assignee_id)
        log.save()
        return Response(WeeklyLogSerializer(log).data)

```

URL: `path('api/logs/<int:pk>/assign/', AssignLogView.as_view(), name='log-assign')`.

Appendix D — Chapter-to-question coverage map

Use this when revising so you don't accidentally skip a lecture.

Lecture	Topic	Practice questions
1. Intro / SDLC	SDLC, workflow states, scoring overview	Q9, Q10, Q11
2. Requirements	User stories, FR/NFR, entities	Q11, Q12
3. System Design	Architecture, ERD, APIs, serializers	Q1, Q2, Q5, Q7, Q12
4. Authentication / RBAC	Authn vs Authz, groups, decorators, React Router	Q13
5. Student Features	Issue form, validation, dashboard	Q2, Q3, Q5, Q7
6. Admin Features	Filtering, assignment, PATCH	Q15
7. Notifications + Workflow	Email, SMS, signals, Toastify	Q4, Q14
Cross-cutting (assured)	Heroku deployment	Q6
Cross-cutting (assured)	Testing	Q8